



Joining Extractions of Regular Expressions

Dominik D. Freydenberger
Loughborough University
Loughborough, United Kingdom
ddfy@ddfy.de

Benny Kimelfeld
Technion –
Israel Institute of Technology
Haifa, Israel
bennyk@cs.technion.ac.il

Liat Peterfreund
Technion –
Israel Institute of Technology
Haifa, Israel
liatpf@cs.technion.ac.il

ABSTRACT

Regular expressions with capture variables, also known as “regex formulas,” extract relations of spans (interval positions) from text. These relations can be further manipulated via the relational Algebra as studied in the context of “document spanners,” Fagin et al.’s formal framework for information extraction. We investigate the complexity of querying text by Conjunctive Queries (CQs) and Unions of CQs (UCQs) on top of regex formulas. Such queries have been investigated in prior work on document spanners, but little is known about the (combined) complexity of their evaluation.

We show that the lower bounds (NP-completeness and $W[1]$ -hardness) from the relational world also hold in our setting; in particular, hardness hits already single-character text. Yet, the upper bounds from the relational world do not carry over. Unlike the relational world, acyclic CQs, and even gamma-acyclic CQs, are hard to compute. The source of hardness is that it may be intractable to instantiate the relation defined by a regex formula, simply because it has an exponential number of tuples. Yet, we are able to establish general upper bounds. In particular, UCQs can be evaluated with polynomial delay, provided that every CQ has a bounded number of atoms (while unions and projection can be arbitrary). Furthermore, UCQ evaluation is solvable with FPT (Fixed-Parameter Tractable) delay when the parameter is the size of the UCQ.

CCS CONCEPTS

• **Theory of computation** → *Models of computation; Formal languages and automata theory; Design and analysis of algorithms; Fixed parameter tractability; Database theory; Data modeling; Database query languages (principles); Database query processing and optimization (theory); Logic and databases*; • **Information systems** → *Information extraction*;

KEYWORDS

Information Extraction; Document Spanners; Regular Expressions; Unions of Conjunctive Queries; Polynomial Delay.

ACM Reference Format:

Dominik D. Freydenberger, Benny Kimelfeld, and Liat Peterfreund. 2018. Joining Extractions of Regular Expressions. In *PODS'18: 35th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems, June 10–15, 2018, Houston, TX, USA*. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3196959.3196967>

1 INTRODUCTION

Information Extraction (IE) conventionally refers to the task of automatically extracting structured information from text. While early work in the area focused largely on military applications [19], this task is nowadays pervasive in a plethora of computational challenges (especially those associated with Big Data), including social media analysis [5], healthcare analysis [38], customer relationship management [3], information retrieval [43], machine log analysis [17], and open-domain Knowledge Base construction [21, 35, 36, 42].

One of the major commercial systems for rule-based IE is IBM’s SystemT that exposes an SQL-like declarative language named AQL (Annotation Query Language), along with a query plan optimizer [31] and development tooling [26]. Conceptually, AQL supports a collection of “primitive” extractors of relations from text (e.g., tokenizer, dictionary lookup, regex matcher and part-of-speech tagger), along with an algebra for relational manipulation (applied to the relations obtained from the primitive extractors). A similar approach is adopted by Xlog [34], where user-defined functions, playing the role of primitive extractors, are manipulated by non-recursive Datalog. DeepDive [32, 35] exposes a declarative language for rules and features, which are eventually translated into the factors of a statistical model where parameters (weights) are set by machine learning. There, the primitive extractors are defined in a scripting language, and the generated relations are again manipulated by relational rules.

Fagin et al. [12] proposed the framework of *document spanners* (or just *spanners* for short) that captures the relational philosophy of the aforementioned systems. Intuitively, a spanner extracts from a document s (which is a string over a finite alphabet) a relation over the spans of s . A *span* of s represents a substring of s that is identified by the start and end indices. An example of a spanner representation is a *regex formula*: a regular expression with embedded capture variables that are viewed as relational attributes. A *regular* spanner is one that can be expressed in the closure of the regex formulas under relational operators projection, union, and join.

For example, it is common to apply sentence boundary detection by evaluating a regex formula [37], which we denote as $\alpha_{\text{sen}}[x]$. When evaluating $\alpha_{\text{sen}}[x]$ over a string s (representing text in natural language), the result is a relation with a single attribute, x , where each tuple consists of the left and right boundaries of a sentence.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

PODS'18, June 10–15, 2018, Houston, TX, USA

© 2018 Association for Computing Machinery.

ACM ISBN 978-1-4503-4706-8/18/06...\$15.00

<https://doi.org/10.1145/3196959.3196967>

The following regular spanner detects sentences that contain both an address in Belgium and the substring police inside them. It assumes a regex formula $\alpha_{\text{adr}}[y, z]$ that extracts (annotates) spans y that represent addresses (e.g., Place de la Nation 2, 1000 Bruxelles, Belgium) with the country being z (e.g., the span of Belgium), the regex formulas $\alpha_{\text{blg}}[z]$ and $\alpha_{\text{plc}}[w]$ that extract spans that are tokens with the words Belgium and police, respectively, and the regex formula $\alpha_{\text{sub}}[y, x]$ that extracts all pairs of spans (x, y) such that y is a subspan of (i.e., has boundaries within) x .

$$\pi_x(\alpha_{\text{sen}}[x] \bowtie \alpha_{\text{adr}}[y, z] \bowtie \alpha_{\text{sub}}[y, x] \bowtie \alpha_{\text{blg}}[z] \bowtie \alpha_{\text{plc}}[w] \bowtie \alpha_{\text{sub}}[w, x]) \quad (1)$$

For example, $\alpha_{\text{sub}}[y, x]$ can be represented as the regex formula

$$\Sigma^* \cdot x \{ \Sigma^* \cdot y \{ \Sigma^* \} \cdot \Sigma^* \} \cdot \Sigma^*$$

where, as we later explain in detail, Σ^* matches every possible string. A key construct in the framework of Fagin et al. [12] is the *variable-set automaton* that was proved that capture precisely the expressive power of regular spanners. A *core* spanner is defined similarly to a regular spanner, but it also allows the string-equality selection predicate on spans (an example can be found further down in this section). Later developments of the framework include the exploration of the complexity of static-analysis tasks on core spanners [16], incorporating inconsistency repairing in spanners [13], a logical characterization [15], and a uniform model for relational data and IE [29].

The research on spanners has been initiated with the motivation of exploring development architectures such as SystemT [25] and Xlog [34]. The basic queries in these architectures are Conjunctive Queries (CQs), and more generally Unions of CQs (UCQs), over regex formulas, expressed as SQL (SystemT) or acyclic Datalog (Xlog). Such UCQs also constitute the language underlying more recent theoretical extensions [13, 29]. Nevertheless, very little is known about the computational complexity of evaluating CQs and UCQs over regex formulas. In this paper, we explore this complexity.

To be more precise, in the framework of (regular and core) spanners, CQs and UCQs over regex formulas are defined similarly to the relational-database world, with two differences. First, the input is not a relational database, but rather a string. Second, the atoms are not relational symbols, but rather regex formulas. Therefore, the *relations* on which the query is applied are implicitly defined as those obtained by evaluating each regex formula over the input string. We refer to these queries as *regex CQs* and *regex UCQs*, respectively. As an example, the above query in (1) is a regex CQ. Under *data complexity*, where the query is assumed fixed, query evaluation is always doable in polynomial time. Here we focus on *combined complexity* where both the query and the data (string) are given as input. One might be tempted to claim that the literature on UCQ evaluation to date should draw the complete picture on complexity, by what we refer to as the *canonical relational evaluation*: evaluate each individual regex formula on the input string, and compute the UCQ as if it were query on an ordinary relational database. There are, however, several problems with this claim.

The first problem is that lower bounds on UCQ evaluation do not carry over immediately to our setting, since the input is not an arbitrary relational database, but rather a very specific one: every relation is obtained by applying a regex to the (same) input string.

But, quite expectedly, we show that the standard lower bound of NP-completeness [7] for Boolean CQs (we well as W[1]-hardness for some parameters) remains in our setting. The more surprising finding is that hardness holds even if the input string is a single character! Yet, a more fundamental problem with the canonical relational approach to evaluation is that it may be infeasible to materialize the relation defined by a regex formula, as the number of tuples in that relation may be exponential in the size of the input. This problem provably increases the complexity, as we show that Boolean regex-CQ evaluation is NP-complete even on acyclic CQs, and even on the more restricted *gamma-acyclic* CQs. In contrast, acyclic CQs (and more generally CQs of *bounded hypertree width* [18]) admit polynomial-time evaluation. Finally, even if we were guaranteed a polynomial bound on the result of each atomic regex formula, it would not necessarily mean that we can actually materialize the corresponding relation in polynomial time.

Yet, in spite of the above daunting complexity we are able to establish some substantial upper bounds. Our upper bounds are based on a central algorithm that we devise in this paper for evaluating a variable-set automaton (*vset-automaton* for short) over a string. More formally, recall that a vset-automaton A represents a spanner, which we denote as $\llbracket A \rrbracket$. When evaluating A on a string s , the result is a relation $\llbracket A \rrbracket(s)$ over the spans of s . The number of tuples in $\llbracket A \rrbracket(s)$ can be exponential in the size of the input (s and A). Our algorithm takes as input a string s and a vset-automaton A (or, more precisely, a *functional* vset-automaton [16]) and enumerates the tuples of $\llbracket A \rrbracket(s)$ with *polynomial delay* [23]. This is done by a nontrivial reduction to the problem of enumerating all the words of a specific length accepted by an NFA [2]. Our central algorithm implies several upper bounds, which we establish in two approaches: (a) via what we call the *canonical relational evaluation*, and (b) via *compilation to automata*.

The first approach utilizes known algorithms for relational UCQ evaluation, by materializing the relations defined by the regex formulas. For that, we devise an efficient compilation of a regex formula into a vset-automaton, and establish that a regex formula can be evaluated in polynomial total time (and even polynomial delay). In particular, we can efficiently materialize the relations of each atom of a regex UCQ, whenever we have a polynomial bound on the cardinality of this relation. Hence, there is no need for a specialized algorithm for each cardinality guarantee—one algorithm fits all. Consequently, under such cardinality guarantees, canonical relational evaluation is efficient whenever the underlying UCQ is tractable (e.g., each CQ is acyclic).

In the second approach, compilation to automata, we compile the entire regex UCQ into a vset-automaton. Combining our polynomial-delay algorithm with known results [12, 15], we conclude that regex UCQs can be evaluated with Fixed-Parameter Tractable (FPT) delay when the size of the UCQ is the parameter. Moreover, we prove that the compilation is efficient *if* every disjunct (regex CQ) has a bounded number atoms. In particular, we show that every join of a bounded number of vset-automata can be compiled in polynomial time into a single vset-automaton, and every projection and union (with no bounds) over vset-automata can be compiled in polynomial time into a single vset-automaton. Hence, we establish that for every fixed k , the evaluation of regex k -UCQs (where each CQ has at most k atoms) can be performed with polynomial delay.

Finally, we generalize our results to allow regex-UCQ to include string-equality predicates, which are expressions that express the relationship “ x and y span the same substring, possibly in different locations”. For example, the following regex CQ with a string equality finds sentences that have the address as expressed in our previous example spanner (see (1) above), but they are not necessarily the same sentences (and in particular they may exclude the word *police*).

$$\pi_{x'} \zeta_{y, y'}^-(Q[w, x, y, z] \bowtie \alpha_{\text{sen}}[x'] \bowtie \alpha_{\text{sub}}[y', x'])$$

where $Q[w, x, y, z]$ is the join expression inside the projection of (1) above. As shown in [16], adding an unbounded number of string equalities can make a tractable regex UCQ intractable, even if that regex UCQ is a single regex formula. We prove that now we no longer retain the above FPT delay, as the problem is $W[1]$ -hard when the parameter is the size of the query.

Nevertheless, much of the two evaluation approaches generalize to string equalities. While this generalization is immediate for the first approach, the second faces a challenge—it is *impossible* to compile string equality into a vset-automaton [12]. Yet, we show that with our compilation techniques, one can compile in string equality *for the specific input string* at hand (that is, not statically but rather at runtime). Hence, we conclude that regex k -UCQs with a bounded number of string equalities can be evaluated with polynomial delay.

The rest of the paper is organized as follows. In Section 2 we recall the framework of document spanners, and set the basic notation and terminology. In Section 3 we give our complexity results for regex UCQs. We describe our polynomial-delay algorithm for evaluating vset-automata in Section 4. In Section 5 we generalize our complexity results to UCQs with string equalities. Finally, we conclude in Section 6.

2 BASICS OF DOCUMENT SPANNERS

We begin by recalling essentials of *document spanners* [12, 13, 16], a formal framework for Information Extraction (IE) inspired by declarative IE systems such as Xlog [34] and IBM's SystemT [24, 25, 31].

2.1 Strings, Spans and Spanners

Throughout this paper, we fix a finite alphabet Σ . Unless explicitly stated, we assume that $|\Sigma| \geq 2$. A *string* is a sequence $\sigma_1 \sigma_2 \dots \sigma_N$ of symbols from Σ . The set of all strings is denoted by Σ^* . We use bold (non-italic) letters, such as $\mathbf{s}$ and $\mathbf{t}$, to denote strings, and ϵ to denote the empty string. The length N of a string $\mathbf{s} := \sigma_1 \sigma_2 \dots \sigma_N$ is denoted by $|\mathbf{s}|$; and $|\mathbf{s}|_\sigma$ denotes the number of occurrences of a symbol σ in $\mathbf{s}$.

Let $\mathbf{s} := \sigma_1 \sigma_2 \dots \sigma_N$ be a string. A *span* of $\mathbf{s}$ represents an interval of characters in $\mathbf{s}$ by indicating the bounding indices. Formally, a span of $\mathbf{s}$ is an expression of the form $[i, j]$ with $1 \leq i \leq j \leq N + 1$. For a span $[i, j]$ of $\mathbf{s}$, we denote by $\mathbf{s}_{[i, j]}$ the string $\sigma_i \dots \sigma_{j-1}$. Note that two spans $[i_1, j_1]$ and $[i_2, j_2]$ are *equal* if and only if both $i_1 = i_2$ and $j_1 = j_2$ hold. In particular, $\mathbf{s}_{[i_1, j_1]} = \mathbf{s}_{[i_2, j_2]}$ does not necessarily imply that $[i_1, j_1] = [i_2, j_2]$.

EXAMPLE 2.1. Our examples assume that Σ consists of the Latin characters and some common additional symbols such as punctuation and commercial at (@). In addition, the symbol $_$ stands for whitespace.

Let $\mathbf{s}$ be the string `chocolate_cookie`. Then $|\mathbf{s}| = 16$. The strings $\mathbf{s}_{[4, 6]}$ and $\mathbf{s}_{[11, 13]}$ are both equal (to the string `co`), and yet, we have $[4, 6] \neq [11, 13]$. Likewise, we observe $\mathbf{s}_{[1, 1]} = \mathbf{s}_{[2, 2]} = \epsilon$, but $[1, 1] \neq [2, 2]$. Finally, note that $\mathbf{s}$ is the same as $\mathbf{s}_{[1, 17]}$. $\diamond$

We assume an infinite set Vars of variables, disjoint from Σ . For a finite $V \subset \text{Vars}$ and $\mathbf{s} \in \Sigma^*$, a $(V, \mathbf{s})$ -tuple is a function μ that maps each variable in V to a span of $\mathbf{s}$. If context allows, we write $\mathbf{s}$ -tuple instead of $(V, \mathbf{s})$ -tuple. A set of $(V, \mathbf{s})$ -tuples is called a $(V, \mathbf{s})$ -relation. A *spanner* is a function P that is associated with a finite variable set V , denoted $\text{Vars}(P)$, and that maps every string $\mathbf{s} \in \Sigma^*$ to a $(V, \mathbf{s})$ -relation $P(\mathbf{s})$.

A spanner P is *Boolean* if $\text{Vars}(P) = \emptyset$. If P is Boolean, then either $P(\mathbf{s}) = \emptyset$ or $P(\mathbf{s})$ contains only the empty $(\emptyset, \mathbf{s})$ -tuple; we interpret these two cases as *false* and *true*, respectively.

2.2 Spanner Representations

In this paper, we use two models as basic building blocks for spanner representations: *regex formulas* and *vset-automata*. The two can be understood as extensions of regular expressions and NFAs, respectively, with variables. Both models were introduced by Fagin et al. [12], and following Freydenberger [15] we define the semantics of these models using so-called *ref-words* [33] (short for *reference-words*).

2.2.1 Ref-words. For a finite variable set $V \subset \text{Vars}$, ref-words are defined over the extended alphabet $\Sigma \cup \Gamma_V$, where Γ_V consists of two symbols, $x\vdash$ and $\vdash x$, for each variable $x \in V$. We assume that Σ and Γ_V are disjoint. Intuitively, the letters $x\vdash$ and $\vdash x$ represent opening or closing a variable x . Hence, ref-words can be understood as terminal strings that are extended with encodings of variable operations. As we shall see, treating these variable operations as letters allows us to adapt techniques from automata theory.

A ref-word $\mathbf{r} \in (\Sigma \cup \Gamma_V)^*$ is *valid for V* if each variable of V is opened and then closed exactly once, or more formally, for each $x \in \text{Vars}(A)$ the string $\mathbf{r}$ has precisely one occurrence of $x\vdash$, precisely one occurrence of $\vdash x$, and the former occurrence takes place before (i.e., to the left of) the latter.

EXAMPLE 2.2. Let $V := \{x\}$, and define the ref-words $\mathbf{r}_1 := c\ x\vdash\ o\ o\ \vdash x\ k\ i\ e$, $\mathbf{r}_2 := x\vdash\ \vdash x$, $\mathbf{r}_3 := \vdash x\ a\ x\vdash$, and $\mathbf{r}_4 := x\vdash\ a\ \vdash x\ x\vdash\ a\ \vdash x$. Then $\mathbf{r}_1$ and $\mathbf{r}_2$ are valid for V , but $\mathbf{r}_3$ and $\mathbf{r}_4$ are not. Note that $\mathbf{r}_1$ and $\mathbf{r}_2$ are not valid for V' with $V' \supset V$, as all variables of V' must be opened and closed. $\diamond$

If V is clear from the context, we simply say that a ref-word is valid. To connect ref-words to terminal strings and later to spanners, we define the clearing morphism $\text{clr} : (\Sigma \cup \Gamma_V)^* \rightarrow \Sigma^*$ by $\text{clr}(\sigma) := \sigma$ for $\sigma \in \Sigma$, and $\text{clr}(a) := \epsilon$ for $a \in \Gamma_V$. For fixed Σ and V , and any $\mathbf{s} \in \Sigma^*$, let $\text{Ref}(\mathbf{s})$ be the set of all valid ref-words $\mathbf{r} \in (\Sigma \cup \Gamma_V)^*$ with $\text{clr}(\mathbf{r}) = \mathbf{s}$. By definition, every $\mathbf{r} \in \text{Ref}(\mathbf{s})$ has a unique factorization

$$\mathbf{r} = \mathbf{r}'_x \cdot x\vdash \cdot \mathbf{r}_x \cdot \vdash x \cdot \mathbf{r}''_x$$

for each $x \in V$. With these factorizations, we interpret $\mathbf{r}$ as a $(V, \mathbf{s})$ -tuple $\mu^{\mathbf{r}}$ by defining $\mu^{\mathbf{r}}(x) := [i, j]$, where $i := |\text{clr}(\mathbf{r}'_x)| + 1$ and

$j := i + |\text{clr}(\mathbf{r}_x)|$. Intuitively, $\text{clr}(\mathbf{r}_x)$ contains $\mathbf{s}_{\mu^r(x)}$, while $\text{clr}(\mathbf{r}'_x)$ contains the prefix of $\mathbf{s}$ to the left.

An alternative way of understanding $\mu^r = [i, j]$ is that i is chosen such that $x \vdash$ occurs between the positions in $\mathbf{r}$ that are mapped to σ_{i-1} and σ_i , and $\neg x$ occurs between the positions that are mapped to σ_{j-1} and σ_j ; assuming that $\mathbf{s} = \sigma_1 \cdots \sigma_{|\mathbf{s}|}$, and slightly abusing the notation to avoid a special distinction for the non-existing positions σ_0 and $\sigma_{|\mathbf{s}|+1}$.

EXAMPLE 2.3. Let $\mathbf{s} := \text{cookie}$, and define $\mathbf{r}_1 := c x \vdash \text{oo} \neg x \text{kie}$, and let $\mathbf{r}_2 := \text{cookie} x \vdash \neg x$. Let $V := \{x\}$. Then $\mathbf{r}_1, \mathbf{r}_2 \in \text{Ref}(\mathbf{s})$, with $\mu^{\mathbf{r}_1}(x) := [2, 4]$ and $\mu^{\mathbf{r}_2}(x) := [6, 7]$. $\diamond$

2.2.2 Regex Formulas. A *regex formula* (over Σ) is a regular expression that may also include variables (called *capture variables*). Formally, we define the syntax with the recursive rule

$$\alpha := \emptyset \mid \epsilon \mid \sigma \mid (\alpha \vee \alpha) \mid (\alpha \cdot \alpha) \mid \alpha^* \mid x\{\alpha\}$$

where $\sigma \in \Sigma$ and $x \in \text{Vars}$. We add and omit parentheses and concatenation ($\cdot$) symbols freely, as long as the meaning remains clear, and use α^+ as shorthand for $\alpha \cdot \alpha^*$, as well as Σ as shorthand for $\bigvee_{\sigma \in \Sigma} \sigma$. The set of variables that occur in α is denoted by $\text{Vars}(\alpha)$. The *size* of α , denoted $|\alpha|$, is naturally defined as the number of symbols in α .

We interpret each regex formula α as a generator¹ of a ref-word language $\mathcal{R}(\alpha)$ over the extended alphabet $\Sigma \cup \Gamma_{\text{Vars}(\alpha)}$. If α is of the form $x\{\beta\}$, then $\mathcal{R}(\alpha) := x \vdash \mathcal{R}(\beta) \neg x$. Otherwise, $\mathcal{R}(\alpha)$ is defined like the language $\mathcal{L}(\alpha)$ of a regular expression; for example, $\mathcal{R}(\alpha \cdot \beta) := \mathcal{R}(\alpha) \cdot \mathcal{R}(\beta)$.

We let $\text{Ref}(\alpha)$ denote the set of all ref-words in $\mathcal{R}(\alpha)$ that are valid for $\text{Vars}(\alpha)$; and for every string $\mathbf{s} \in \Sigma^*$, we define $\text{Ref}(\alpha, \mathbf{s}) = \text{Ref}(\alpha) \cap \text{Ref}(\mathbf{s})$. In other words, $\text{Ref}(\alpha, \mathbf{s})$ contains exactly those valid ref-words from $\mathcal{R}(\alpha)$ that clr maps to $\mathbf{s}$.

Finally, the spanner $\llbracket \alpha \rrbracket$ is the one that defines the following $(\text{Vars}(\alpha), \mathbf{s})$ -relation for every string $\mathbf{s} \in \Sigma^*$:

$$\llbracket \alpha \rrbracket(\mathbf{s}) := \{\mu^r \mid \mathbf{r} \in \text{Ref}(\alpha, \mathbf{s})\}$$

We say that a regex-formula α is *functional* if $\mathcal{R}(\alpha) = \text{Ref}(\alpha)$; that is, every ref-word in $\mathcal{R}(\alpha)$ is valid. Fagin et al. [12] gave an algorithm for testing whether a regex formula is functional. By analyzing the complexity of their construction, we conclude the following.

THEOREM 2.4. [12] *Whether a regex formula α with v variables is functional can be tested in $O(|\alpha| \cdot v)$ time.*

Following [12], we assume regex formulas are functional unless explicitly noted (in fact, [12] does not even define $\llbracket \alpha \rrbracket$ for non-functional regex formulas α).

EXAMPLE 2.5. Consider the following regex formula α .

$$\Sigma^* ((x\{\text{foo}\} \Sigma^* y\{\text{bar}\}) \vee (y\{\text{bar}\} \Sigma^* x\{\text{foo}\})) \Sigma^*$$

Then $\text{Vars}(\alpha) = \{x, y\}$. For $\mathbf{s} \in \Sigma^*$, the $(\text{Vars}(\alpha), \mathbf{s})$ -relation $\llbracket \alpha \rrbracket(\mathbf{s})$ contains every $(\text{Vars}(\alpha), \mathbf{s})$ -tuple μ that satisfies $\mathbf{s}_{\mu(x)} = \text{foo}$ and $\mathbf{s}_{\mu(y)} = \text{bar}$. Now, consider the regex formula β defined as follows.

$$\Sigma^* \text{mail}\{x_{\text{user}}\{Y\} @ x_{\text{domain}}\{Y \cdot Y\}\} \Sigma^*,$$

¹Note that we can also interpret each regex formula α (and, in fact, all spanners) generators of a (terminal) language $\mathcal{L}(\alpha)$ over Σ , see [15, 16] for details. Keeping this in mind, the use of $\mathcal{R}$ is intended to highlight that words from ref-languages usually contain additional symbols that do not belong to Σ .

where $Y := (a \vee \cdots \vee z)^*$, and, as previously said, $\text{ }_\text{ }_\text{ }$ denotes whitespace. This regex formula identifies (simplified) email addresses, where the variable x_{mail} contains the whole address, and x_{user} and x_{domain} the user and domain parts. Both α and β are functional. In contrast, $x\{a\} x\{a\}$ and $x\{a\} \vee y\{a\}$ are not functional. $\diamond$

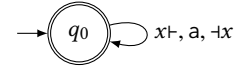
2.2.3 Variable-Set Automata. While the current paper is mostly motivated by spanners that are constructed using regex formulas, our upper bounds for these are obtained by converting regex formulas to the following automata model.

A *variable-set automaton* (vset-automaton for short) with variables from a finite set $V \subset \text{Vars}$ can be understood as an ϵ -NFA (i.e., an NFA with epsilon transitions allowed) that is extended with edges labeled with variable operations $x \vdash$ or $\neg x$ for $x \in V$. Formally, a vset-automaton is a tuple $A := (V, Q, q_0, q_f, \delta)$, where V is a finite set of variables, Q is the set of states, $q_0, q_f \in Q$ are the *initial* and the *final* states, respectively, and $\delta: Q \times (\Sigma \cup \{\epsilon\} \cup \Gamma_V) \rightarrow 2^Q$ is the *transition function*. By $\text{Vars}(A)$ we denote the set V .

The vset-automaton $A := (V, Q, q_0, q_f, \delta)$ can be interpreted as a directed graph, where the nodes are the states, and every $q \in \delta(p, a)$ is represented by an edge from p to q with the label a . To define the semantics of A , we first interpret A as an ϵ -NFA over the terminal alphabet $\Sigma \cup \Gamma_V$, and define its *ref-word language* $\mathcal{R}(A)$ as the set of all ref-words $\mathbf{r} \in (\Sigma \cup \Gamma_V)^*$ such that some path from q_0 to q_f is labeled with $\mathbf{r}$.

Let A be a vset-automaton. Analogously to regex formulas, we denote by $\text{Ref}(A)$ the set of all ref-words in $\mathcal{R}(A)$ that are valid for V , and define $\text{Ref}(A, \mathbf{s})$ and $\llbracket A \rrbracket(\mathbf{s})$ accordingly for every $\mathbf{s} \in \Sigma^*$. Likewise, we say that A is *functional* if $\text{Ref}(A) = \mathcal{R}(A)$; i.e., every accepting run of A generates a valid ref-word. Two vset-automata A_1 and A_2 are *equivalent* if $\llbracket A_1 \rrbracket = \llbracket A_2 \rrbracket$.

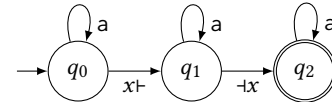
EXAMPLE 2.6. Let A be the following vset-automaton:



Then $\mathcal{R}(A) = \{x \vdash, a, \neg x\}^*$, and

$$\text{Ref}(A) = \{a^i x \vdash a^j \neg x a^k \mid i, j, k \geq 0\}.$$

Hence, A is not functional, since $\mathcal{R}(A)$ contains invalid ref-words such as ϵ , $x \vdash$, $\neg x a x \vdash$, and $x \vdash a \neg x \neg x$. Now consider the following vset-automaton A_{fun} :



Then $\text{Ref}(A_{\text{fun}}) = \mathcal{R}(A_{\text{fun}}) = \text{Ref}(A)$. Hence, A_{fun} is functional, and A_{fun} and A are equivalent.

For all $\mathbf{s} \in \{a\}^*$, we have that $\llbracket A \rrbracket(\mathbf{s})$ contains all possible $(\{x\}, \mathbf{s})$ -tuples. Moreover, for all $\mathbf{s} \in \Sigma^* \setminus \{a\}^*$, we observe $\llbracket A \rrbracket(\mathbf{s}) = \emptyset$. $\diamond$

While $\text{Ref}(A_1) = \text{Ref}(A_2)$ is a sufficient criterion for the equivalence of A_1 and A_2 , it is only characteristic if the automata have at most one variable. For example, consider $\mathbf{r}_1 := x \vdash y \vdash \neg x \neg y$ and $\mathbf{r}_2 := y \vdash \neg y x \vdash \neg x$. Both are valid ref-words that define the same $(\{x, y\}, \epsilon)$ -tuple $[1, 1]$, but $\mathbf{r}_1 \neq \mathbf{r}_2$.

Example 2.6 suggests that vset-automata can be converted into equivalent functional vset-automata. However, Freydenberger [15]

showed that although this conversion is possible with standard automata constructions, the resulting blow-up may be exponential in the number of variables.

As shown in Lemma 3.4, every functional regex formula can be converted into an equivalent functional vset-automaton; and we use this connection throughout the paper. We shall see that functional vset-automata are a convenient tool for working with regex formulas. In contrast to this, vset-automata in general can be quite inconvenient (e. g., even deciding emptiness of $\llbracket A \rrbracket(\epsilon)$ is NP-complete if A is not functional, see [15]).

Freydenberger [15] established the complexity of testing whether a given vset-automaton is functional.

THEOREM 2.7. [15] *Whether a given vset-automaton α with n states, m transitions and v variables is functional can be tested in $O(vm + n)$ time.*

A special property of functional vset-automata is that each state implicitly stores which variables have been opened and closed. We discuss it in detail in Section 4.2.

2.2.4 Spanner Algebras. Let P, P_1 and P_2 be spanners. The algebraic operators *union*, *projection*, *natural join*, and *selection* are defined as follows:

Union: If $\text{Vars}(P_1) = \text{Vars}(P_2)$, their *union* $(P_1 \cup P_2)$ is defined by $\text{Vars}(P_1 \cup P_2) := \text{Vars}(P_1)$ and $(P_1 \cup P_2)(s) := P_1(s) \cup P_2(s)$ for all $s \in \Sigma^*$.

Projection: Let $Y \subseteq \text{Vars}(P)$. The *projection* $\pi_Y P$ is defined by $\text{Vars}(\pi_Y P) := Y$ and $\pi_Y P(s) := P|_Y(s)$ for all $s \in \Sigma^*$, where $P|_Y(s)$ is the restriction of all $\mu \in P(s)$ to Y .

Natural join: Let $V_i := \text{Vars}(P_i)$ for $i \in \{1, 2\}$. The *join* $(P_1 \bowtie P_2)$ of P_1 and P_2 is defined by $\text{Vars}(P_1 \bowtie P_2) := \text{Vars}(P_1) \cup \text{Vars}(P_2)$ and, for all $s \in \Sigma^*$, $(P_1 \bowtie P_2)(s)$ is the set of all $(V_1 \cup V_2, s)$ -tuples μ for which there exist $\mu_1 \in P_1(s)$ and $\mu_2 \in P_2(s)$ with $\mu|_{V_1}(s) = \mu_1(s)$ and $\mu|_{V_2}(s) = \mu_2(s)$.

Selection: Let $x_1, \dots, x_k \in \text{Vars}(P)$. We define the *string equality selection* $\zeta_{x_1, \dots, x_k}^- P$ by $\text{Vars}(\zeta_{x_1, \dots, x_k}^- P) := \text{Vars}(P)$ and, for all $s \in \Sigma^*$, we have that $\zeta_{x_1, \dots, x_k}^- P(s)$ is the set of all $\mu \in P(s)$ for which $s_{\mu(x_1)} = \dots = s_{\mu(x_k)}$ holds.

Note that the join operator joins tuples that have identical spans in their shared variables. In contrast, the selection operator compares the substrings of s that are described by the spans, and does not distinguish between different spans that span the same substrings.

Following Fagin et al. [12], we refer to regex formulas and vset-automata as *primitive spanner representations*, and use VA_{set} and RGX to denote the sets of all vset-automata and all functional regex formulas, respectively. A *spanner algebra* is a finite set of spanner operators. If $\mathcal{O}$ is a spanner algebra and $\mathcal{C}$ is a class of primitive spanner representations, then $\mathcal{C}^{\mathcal{O}}$ denotes the set of all *spanner representations* that can be constructed by (repeated) combination of the symbols for the operators from $\mathcal{O}$ with spanners from $\mathcal{C}$. For each spanner representation of the form oq or $q_1 \circ q_2$, where $o \in \mathcal{O}$ and $q, q_1, q_2 \in \mathcal{C}$, we define $\llbracket oq \rrbracket = o[\llbracket q \rrbracket]$ and $\llbracket q_1 \circ q_2 \rrbracket = \llbracket q_1 \rrbracket \circ \llbracket q_2 \rrbracket$. Furthermore, $\llbracket \mathcal{C}^{\mathcal{O}} \rrbracket$ is the closure of $\llbracket \mathcal{C} \rrbracket$ under the spanner operators in $\mathcal{O}$.

Fagin et al. [12] refer to the elements of $\text{RGX}^{\{\pi, \cup, \bowtie\}}$ as *regular spanner representations*, and to the elements of $\text{RGX}^{\{\pi, \zeta^-, \cup, \bowtie\}}$ as *core spanner representations* (as these form the core of the query language AQL [25]). One of the main results of [12] is that

$$\llbracket \text{RGX}^{\{\pi, \cup, \bowtie\}} \rrbracket = \llbracket \text{VA}_{\text{set}}^{\{\pi, \cup, \bowtie\}} \rrbracket = \llbracket \text{VA}_{\text{set}} \rrbracket.$$

Hence, $\text{VA}_{\text{set}}^{\{\pi, \zeta^-, \cup, \bowtie\}}$ has the same expressive power as core spanner representations. A *regular spanner* is a spanner that can be represented in a regular spanner representation, and a *core spanner* is a spanner that can be represented in a core spanner representation. Fagin et al. [12] also showed the class of regular spanners is closed under the *difference* operator (i.e., $\llbracket \text{RGX}^{\{\pi, \cup, \bowtie\}} \rrbracket = \llbracket \text{RGX}^{\{\pi, \cup, \bowtie, \setminus\}} \rrbracket$), but the class of core spanners is not.

2.3 (Unions of) Conjunctive Queries

In this paper, we consider Conjunctive Queries (CQs) over regex formulas. Such queries are defined as the class of regular spanner representations that can be composed out of natural join and projection. In addition, we explore the extension obtained by adding string-equality selections.

More formally, a *regex CQ* is an regular spanner representation of the form $q := \pi_Y (\alpha_1 \bowtie \dots \bowtie \alpha_k)$ where each α_i is a regex formula. A *regex CQ with string equalities* is a core spanner representation of the form

$$q := \pi_Y (\zeta_{x_1, x_2}^- \dots \zeta_{x_l, x_{l+1}}^- (\alpha_1 \bowtie \dots \bowtie \alpha_k))$$

for some $k \geq 1$ and $l \geq 0$. For clarity of notation, if $Y = \{y_1, \dots, y_m\}$, we sometimes write $q(y_1, \dots, y_m)$ when using q to make $\text{Vars}(q)$ more explicit. Each regex formula α_i and each selection ζ_{x_j, y_j}^- is called an *atom*, where the former is a *regex atom* and the latter an *equality atom*. We denote by $\text{atoms}(q)$ the set of atoms of q . For each equality atom ζ_{x_j, y_j}^- , we define $\text{Vars}(\zeta_{x_j, y_j}^-) := \{x_j, y_j\}$. Note that our definitions imply that every variable that occurs in an equality atom also appears in at least one regex atom.

In the traditional relational model (see, e.g., Abiteboul et al. [1]), a CQ is phrased over a collection of relation symbols (called *signature*), each having a predefined arity. Formally, a *relational CQ* is an expression of the form $Q(y_1, \dots, y_m) := \varphi_1, \dots, \varphi_k$ where each y_i is variable in Vars and each φ_i is an *atomic relational formula* (or simply *atom*), that is, an expression of the form $R(x_1, \dots, x_m)$ where R is an m -ary relation symbol and each x_j is a variable. Similarly to regex CQs, we denote by $\text{atoms}(Q)$ the set of atoms of Q , and by $\text{Vars}(Q)$ the set of variables that occur in an atom Q .

Let $q(y_1, \dots, y_m)$ be a regex CQ (with string equalities), and let $Q(y_1, \dots, y_m)$ be a relational CQ. We say that q *maps to* Q if all of the following hold.

- No relation symbol occurs more than once in Q (that is, Q has no self joins).
- There is a bijection $\mu : \text{atoms}(q) \rightarrow \text{atoms}(Q)$ that preserves the sets of variables, that is, for each $\gamma \in \text{atoms}(q)$ we have $\text{Vars}(\gamma) = \text{Vars}(\mu(\gamma))$.

Let q be a regex CQ with string equalities. We say that q is *acyclic* if it maps to an acyclic (or *alpha-acyclic*) relational CQ, and *gamma-acyclic* if it maps to a gamma-acyclic relational CQ. (See e.g. [1, 11] for the definitions of acyclicity.) Recall that gamma-acyclicity is

strictly more restricted than acyclicity (that is, every gamma-acyclic CQ is acyclic, and there are acyclic CQs that are not gamma-acyclic).

A *Union of regex Conjunctive Queries*, or *regex UCQ* for short, is a regular spanner q of the form $q := \bigcup_{i=1}^k q_i$ where each q_i is a regex CQ (recall that, by definition, $\text{Vars}(q_i) = \text{Vars}(q_j)$ must hold). In a regex UCQ *with string equalities*, each q_i can be a UCQ with string equalities. The following theorem follows quite easily from the results of Fagin et al. [12].

THEOREM 2.8. *The following hold.*

- *The class of spanners expressible as regex UCQs is that of the regular spanners.*
- *The class of spanners expressible as regex UCQs with string equalities is that of the core spanners.*

In the relational world, a *relational UCQ* is a query of the form $\bigcup_{i=1}^l Q_i$, where each Q_i is a relational CQ and $\text{Vars}(Q_i) = \text{Vars}(Q_j)$ holds. Given a regex UCQ (with or without string equalities)

$$q := \bigcup_{i=1}^k q_i(y_1, \dots, y_m)$$

and a relational UCQ

$$Q := \bigcup_{i=1}^l Q_i(y_1, \dots, y_m),$$

we say that q *maps to* Q if $k = l$ and each q_i maps to Q_i .

A family $\mathbf{P}$ of regex UCQs (with string equalities) *maps to* a family $\mathbf{Q}$ of relational UCQs if each UCQ in $\mathbf{P}$ maps to one or more CQ in $\mathbf{Q}$.

3 COMPLEXITY OF UCQ EVALUATION

In this section we give our main complexity results for the evaluation of regex UCQs. We remark that in this section we do not allow string equalities; these will be discussed in Section 5. We begin with a description of the complexity measures we adopt.

3.1 Complexity Measures

Under the measure of *data complexity*, where the UCQ q at hand is assumed fixed (and the string s is given as input), it is a straightforward observation that query evaluation can be done in polynomial time (see Freydenberger and Holldack [16]). Hence, our main measure of complexity is that of *combined complexity* where both q and s are given as input.

The task of evaluating a regex query q over an input s requires the solver algorithm to produce all tuples in $\llbracket q \rrbracket(s)$ for each regex formula γ in q . In the worst case there could be exponentially many tuples, and so *polynomial time* is not a proper yardstick of efficiency. For such problems, Johnson, Papadimitriou and Yannakakis [23] introduced several complexity guarantees, which we recall here. An *enumeration problem* $\mathbf{E}$ is a collection of pairs (x, Y) where x is an *input* and Y is a finite set of *answers* for x , denoted by $\mathbf{E}(x)$. In our case, x has the form (q, s) and $\mathbf{E}(x)$ is $\llbracket q \rrbracket(s)$. A *solver* for an enumeration problem $\mathbf{E}$ is an algorithm that, when given an input x , produces a sequence of answers such that every answer in $\mathbf{E}(x)$ is printed precisely once. We say that a solver S for an enumeration problem $\mathbf{E}$ runs in:

- *polynomial total time* if the total execution time of S is polynomial in $(|x| + |\mathbf{E}(x)|)$;
- *polynomial delay* if the time between every two consecutive answers produced is polynomial in $|x|$.

We also consider *parameterized complexity* [10, 20] for various parameters determined by q . Formally, a *parameterized problem* is a decision problem where the input consists of a pair (x, k) , where x is an ordinary input and k is a parameter (typically small, relates to a property of x). Such a problem is *Fixed-Parameter Tractable (FPT)* if there is a polynomial p , a computable function f and a solver S , such that S terminates in time $f(k) \cdot p(|x|)$ on input (x, k) . We similarly define *FPT-delay* for a parameterized enumeration algorithm: the delay between every two consecutive answers is bounded by $f(k) \cdot p(|x|)$. A standard lower bound is *W[1]-hardness*, and the standard complexity assumption is that a *W[1]-hard* problem is not FPT [20].

Whenever we give an upper bound, it applies to a general UCQ, and whenever we give a lower bound, it applies to Boolean CQs. When we give asymptotic running times, we assume the unit-cost RAM-model, where the size of each machine word is logarithmic in the size of the input. Regarding Σ , our lower bounds and asymptotic upper bounds assume that it is fixed with at least two characters; our “polynomial” upper bounds hold even if Σ is given as part of the input.

3.2 Lower Bounds

We begin with lower bounds. Recall that Boolean CQ evaluation is NP-complete [7]. This result does not extend directly to regex UCQs, since relations are not given directly as input (but rather extracted from an input string using regex formulas). But quite expectedly, the evaluation of Boolean regex CQs indeed remains NP-complete. What is less expected is that this holds even for strings that consist of a single fixed character. That is, we show that the evaluation of Boolean regex CQs is NP-complete also under the measure of *query complexity*, where the UCQ q is given as input (and s is fixed).

THEOREM 3.1. *Evaluation of Boolean regex CQs is NP-complete, and remains NP-hard even under both of the following assumptions.*

- (1) *Each regex formula is of bounded size.*
- (2) *The input string is of length one.*

PROOF. The upper bound is obvious (even for core spanners, see [16]). For the lower bound, we construct a reduction from 3CNF-satisfiability to the evaluation problem of Boolean regex CQs. The input to 3CNF is a formula ψ with the free variables $x_1, \dots, x_n$ such that ψ has the form $C_0 \wedge \dots \wedge C_m$ where each C_j is a clause. Each clause is a conjunction of three literals from the set

$$\{x_i, \neg x_i \mid 1 \leq i \leq n\}.$$

The goal is to determine whether there is an assignment τ from $\{x_1, \dots, x_n\}$ to $\{0, 1\}$ that satisfies ψ . Given a 3CNF-formula ψ , we construct a regex CQ q and an input string s such that there is a satisfying assignment for ψ if and only if $\llbracket q \rrbracket(s) \neq \emptyset$. We define $s := a$. To construct q , we associate each variable x with a corresponding capture variable x and each assignment τ with a regex formula that assigns x the span $[1, 1]$ if $\tau(x) = 0$ and $[2, 2]$ if $\tau(x) = 1$. For instance, given the assignment τ such that $\tau(x) =$

$\tau(y) = 0$ and $\tau(z) = 1$ its corresponding regex is $x\{y\{\epsilon\}\} \cdot a \cdot z\{\epsilon\}$. Note that since each clause C_i contains three variables, it has exactly seven satisfying assignments (out of all possible eight assignments to its variables). We denote these assignments by $\tau_i^1, \dots, \tau_i^7$ and their corresponding regex formulas by $\gamma_i^1, \dots, \gamma_i^7$. Next, we define γ_{α_i} and q as follows.

$$\gamma_i := \bigvee_{j=1}^7 \gamma_i^j \quad q := \pi_{\emptyset} \bigotimes_{i=1}^m \gamma_i$$

It is straightforward to show that if there exists a satisfying assignment τ for ψ then there is at least one $\mu \in \llbracket q \rrbracket(s)$. This μ is obtained from τ by defining $\mu(x) := [1, 1]$ if $\tau(x) = 0$ and $\mu(x) := [2, 2]$ if $\tau(x) = 1$. The other direction is shown analogously: If there is at least one $\mu \in \llbracket q \rrbracket(s)$, then a satisfying assignment τ for ψ is obtained by defining $\tau(x) := 0$ if $\mu(x) = [1, 1]$ and $\tau(x) := 1$ if $\mu(x) = [2, 2]$. $\square$

One might be tempted to think that the evaluation of regex CQs over a string s is tractable as long as the regex CQ

$$q := \pi_Y(\alpha_1 \bowtie \dots \bowtie \alpha_k)$$

maps to a relational CQ of a tractable class (e.g., acyclic CQs where evaluation is in polynomial total time [41]), by applying what we refer to as the *canonical relational evaluation*:

- (a) Evaluate each regex formula: $r_i := \llbracket \alpha_i \rrbracket(s)$.
- (b) Evaluate $\pi_Y(r_1 \bowtie \dots \bowtie r_k)$ (as a relational CQ).

There are, though, two problems with the canonical relational evaluation. The first (and main) problem is that r_i may already be too large (e.g., exponential number of tuples). The second problem is that, even if r_i is of manageable size, it is not clear that it can be efficiently constructed. In the next section we will show that the second problem is solvable: we can evaluate α_i over s in polynomial total time. However, the first problem remains. In fact, the following theorem states that the evaluation of regex CQs is intractable, even if we restrict to ones that map to acyclic CQs, and even the more restricted gamma-acyclic CQs! In addition, we can show $W[1]$ -hardness with respect to the number of variables or regex formulas.

THEOREM 3.2. *Evaluation of gamma-acyclic Boolean regex CQs is NP-complete. The problem is also $W[1]$ -hard with respect to the number of (a) variables, and (b) atoms.*

PROOF. The NP-upper bound was already discussed in the proof of Theorem 3.1. We prove both lower bounds at the same time by defining a polynomial time FPT-reduction from the k -clique problem. Given an undirected graph $G := (V, E)$ and a $k \geq 2$, this problem asks whether G contains a clique with k nodes. Let $\Sigma := \{a, b, \vdash, \#, \dashv\}$ (the proof can be adapted to a binary Σ with standard techniques). We assume $V = \{v_1, \dots, v_n\}$, $n = |V|$, and associate each $v_i \in V$ with a unique string $\mathbf{v}_i \in \{a, b\}^*$ such that $|\mathbf{v}_i|$ is $O(\log n)$.

We define $\mathbf{s} := (\mathbf{e}_{1,2} \cdots \mathbf{e}_{1,n}) \cdot (\mathbf{e}_{2,3} \cdots \mathbf{e}_{2,n}) \cdots (\mathbf{e}_{n-1,n})$, where

$$\mathbf{e}_{i,j} := \begin{cases} \epsilon & \text{if } \{v_i, v_j\} \notin E, \\ \vdash \mathbf{v}_i \# \mathbf{v}_j \dashv & \text{if } \{v_i, v_j\} \in E. \end{cases}$$

for all $1 \leq i < j \leq n$. Thus, $\mathbf{s}$ encodes E , such that an edge $\{v_i, v_j \mid i < j\}$ precedes an edge $\{v_{i'}, v_{j'} \mid i' < j'\}$ if $i < i'$, or $i = i'$ and $j < j'$.

Next, we construct q such that $\llbracket q \rrbracket(s) \neq \emptyset$ if and only if there is a k -clique in G . Note that a k -clique has k nodes $v_{c(1)}, \dots, v_{c(k)}$, and we assume that $i < j$ implies $c(i) < c(j)$. For each $v_{c(l)}$, q shall contain the $(k-1)$ variables $y_{1,l}$ to $y_{l-1,l}$ and $x_{l,l+1}$ to $x_{l,k}$. The intuition is that each pair $x_{i,j}$ and $y_{i,j}$ of variables corresponds to an edge $\{v_{c(i)}, v_{c(j)}\}$. In particular, $x_{i,j}$ and $y_{i,j}$ shall represent the node with the smaller and larger index, respectively. To this end, we define $\gamma := \gamma_1 \cdots \gamma_{k-1}$ with $\gamma_i := \gamma_{i,i+1} \cdots \gamma_{i,k}$ and

$$\gamma_{i,j} := \Sigma^* \vdash x_{i,j} \{(a \vee b)^*\} \# y_{i,j} \{(a \vee b)^*\} \dashv \Sigma^*$$

for all $1 \leq i < j \leq k$. The idea is that $x_{i,j}$ and $y_{i,j}$ respectively match $\mathbf{v}_{c(i)}$ and $\mathbf{v}_{c(j)}$ in $\mathbf{s}$, which uses the same order. We now want to ensure that for each $1 \leq l \leq k$, all $y_{i,l}$ and all $x_{l,j}$ with $1 \leq i < l < j \leq k$ have to be matched to various occurrences of the same substring. To ensure this, for each $1 \leq l \leq k$, we define the regex formula $\delta_l := \bigvee_{i=1}^n \delta_{l,\mathbf{v}_i}$ where

$$\begin{aligned} \delta_{l,\mathbf{v}} := & \Sigma^* \# y_{1,l} \{\mathbf{v}\} \dashv \Sigma^* \cdots \Sigma^* \# y_{l-1,l} \{\mathbf{v}\} \dashv \Sigma^* \\ & \cdot \Sigma^* \vdash x_{l,l+1} \{\mathbf{v}\} \# \Sigma^* \cdots \Sigma^* \vdash x_{l,k} \{\mathbf{v}\} \# \Sigma^*. \end{aligned}$$

Finally we define q to be the query

$$q := \pi_{\emptyset} \left(\gamma \bowtie \bigotimes_{1 \leq i \leq k-1} \delta_i \right)$$

Note that q contains $O(k)$ atoms and $O(k^2)$ variables. Additionally, q contains no gamma-cycles since each two different δ_l have no common variables. Moreover, as $|\gamma|$ is $O(k^2)$, and each $|\delta_l|$ is $O(kn \log n)$, $|q|$ is $O(k^2 + k^2 n \log n) = O(k^2 n \log n)$. Furthermore, $|\mathbf{s}|$ is $O(|E|)$. Hence, q and $\mathbf{s}$ can be constructed in polynomial time, and the construction is FPT with respect to the number of variables and atoms.

All that is left to show that the reduction is correct; that is $\llbracket q \rrbracket(s) \neq \emptyset$ iff. G contains a k -clique. Assume that G contains a k -clique $\{v_{c(1)}, \dots, v_{c(k)}\}$ where $c(i) < c(j)$ whenever $i < j$. Let μ be an s -tuple that is defined as follows: For all $1 \leq i < j \leq k$, find the substring $\vdash \mathbf{v}_{c(i)} \# \mathbf{v}_{c(j)} \dashv$. Then map $x_{i,j}$ to the span that corresponds to $\mathbf{v}_{c(i)}$ in this substring, and $y_{i,j}$ to the span that corresponds to $\mathbf{v}_{c(j)}$. Then $\mu \in \llbracket \gamma \rrbracket(s)$, since each two nodes in the clique are connected and since the encoding of E in $\mathbf{s}$ is ordered. Moreover, for each l , the restriction of μ to $\text{Vars}(\delta_l)$ is in $\llbracket \delta_l \rrbracket(s)$, since the strings spanned by the $y_{i,l}$ and the $x_{l,j}$ are equal, and μ respects the order of the variables in δ_l .

Now assume $\mu \in \llbracket q \rrbracket(s)$. We can now derive the nodes $v_{c(1)}$ to $v_{c(k)}$ of the clique directly from the variables $x_{i,j}$ and $y_{i,j}$, as for each l , $\mu \in \llbracket \delta_l \rrbracket(s)$ ensures that there is a unique $c(l)$ such that $\mathbf{s}_{\mu(y_{i,l})} = \mathbf{s}_{\mu(x_{l,j})} = \mathbf{v}_{c(l)}$ for all $1 \leq i < l < j \leq k$. Furthermore, $\mu \in \llbracket \gamma \rrbracket(s)$ ensures that for all $i < j$, $\mu(x_{i,j})$ and $\mu(y_{i,j})$ map to the encoding of the edge $\{v_{c(i)}, v_{c(j)}\}$ in $\mathbf{s}$. Hence, $\{v_{c(1)}, \dots, v_{c(k)}\}$ is a k -clique in G . $\square$

Note that we did not show $W[1]$ -hardness with respect to the length of the query.

3.3 Upper Bounds

3.3.1 Evaluation of Variable-Set-Automata. Our central complexity result states that functional vset-automata can be evaluated with polynomial delay.

THEOREM 3.3. *Given a functional vset-automaton A with n states and m transitions, and a string s , one can enumerate $\llbracket A \rrbracket(s)$ with polynomial delay of $O(n^2|s|)$, following a polynomial preprocessing of $O(n^2|s| + mn)$.*

Note that Losemann [27] introduced an algorithm that evaluates vset-automata with logarithmic delay, but only if the number of variables is bounded.² We discuss our algorithm and other details of the proof in Section 4. In the remainder of this section, we explain the implications of this theorem for both evaluation approaches.

3.3.2 Canonical Relational Approach. It was already shown by Fagin et al. [12] that every regex formula can be converted into an equivalent vset-automaton (where a vset-automaton A and a regex formula α are said to be *equivalent* if $\llbracket A \rrbracket = \llbracket \alpha \rrbracket$). It is probably not at all surprising that this is possible in a way that is efficient and results in functional vset-automata (recall that we assume regex formulas to be functional by convention).

LEMMA 3.4. *Given a regex formula α , one can construct in time $O(|\alpha|)$ a functional vset-automaton A with $\llbracket A \rrbracket = \llbracket \alpha \rrbracket$.*

PROOF. Let α be a regex formula, and define $V := \text{Vars}(\alpha)$. Assume that α is represented as its syntax tree. We first rewrite α into a regular expression $\hat{\alpha}$ over the alphabet $\Sigma \cup \Gamma_V$ with $\mathcal{L}(\hat{\alpha}) = \mathcal{R}(\alpha)$. This is done by recursively replacing every node that represents a variable binding $x\{\beta\}$ with the concatenation $x\vdash\beta\cdot\vdash x$. Then the length of $\hat{\alpha}$ is linear in the length of α ; and the rewriting is possible in linear time as well.

Next, we convert $\hat{\alpha}$ into an ϵ -NFA A with $\mathcal{L}(A) = \mathcal{L}(\hat{\alpha})$. Using the Thompson construction (cf. e.g., [22]), this is possible in linear time. Furthermore, both the number of states and the number of transitions of A are linear in the length of $\hat{\alpha}$ (and, hence, also in the length of α). Finally, note that the construction ensures that A has only one accepting state (recall that vset-automata are required to have a single accepting state).

This allows us to interpret A as a vset-automaton with variable set V , using $\mathcal{R}(A) = \mathcal{L}(A)$. As $\mathcal{L}(A) = \mathcal{L}(\hat{\alpha}) = \mathcal{R}(\alpha)$ holds by definition, we know that $\mathcal{R}(A) = \mathcal{R}(\alpha)$. Furthermore, as α is functional, $\mathcal{R}(\alpha) = \text{Ref}(\alpha)$ holds, which implies $\text{Ref}(A) = \mathcal{R}(A) = \text{Ref}(\alpha)$. This allows us to make two conclusions: Firstly, that $\text{Ref}(A) = \mathcal{R}(A)$, hence A is functional. And, secondly, that $\llbracket A \rrbracket = \llbracket \alpha \rrbracket$, as $\text{Ref}(A) = \text{Ref}(\alpha)$ implies $\text{Ref}(A, s) = \text{Ref}(\alpha, s)$ for all $s \in \Sigma^*$. $\square$

We note that the result of Lemma 3.4 was shown independently by Morciano et al. [28].

We note here that the complexity of the preprocessing stated in Theorem 3.3 holds for a general functional vset-automaton. If, however, the automaton is derived from a regex formula α by the

²Otherwise, two steps of the algorithm become exponential, namely the encoding of vset-automata to another automata model and the actual enumeration. Using functional automata might make the first step polynomial, but not the second.

construction we use for proving Lemma 3.4, then the time of this preprocessing drops to $O(n^2|s|)$, where n is $O(|\alpha|)$.

As a consequence, we conclude that the canonical relational approach to evaluation is actually efficient for UCQs, under two conditions. The first condition is that the regex CQs map to a tractable class of relational CQs. More formally, by *tractable class* of relational CQs we refer to a class $\mathcal{Q}$ of CQs that can be evaluated in polynomial total time,³ such as acyclic CQs, or more generally CQs with *bounded hypertree width* [18]. But, as shown in Theorem 3.2, this condition is not enough. The second condition is that there is a polynomial bound on the number of tuples of each regex formula. More formally, we say that a class $\mathcal{A}$ of regex formulas is *polynomially bounded* if there exists a positive integer d such that for every regex formula $\alpha \in \mathcal{A}$ and string s we have $|\llbracket \alpha \rrbracket(s)| = O(|s|^d)$.

Clearly, if every regex formula in $\mathcal{A}$ can be evaluated in polynomial time, then $\mathcal{A}$ is polynomially bounded. From Theorem 3.3 we conclude that the other direction also holds. Hence, from Theorem 3.3 and Lemma 3.4 we establish the following theorem.

THEOREM 3.5. *Let $\mathcal{P}$ be a class of regex UCQs. If the regex formulas in the UCQs of $\mathcal{P}$ belong to a polynomially bounded class, and $\mathcal{P}$ maps to a tractable class of relational UCQs, then UCQs in $\mathcal{P}$ can be evaluated in polynomial total time.*

Examples of polynomially bounded classes of regex formulas are the following.

- The class of regex formulas with at most k variables, for some fixed k .
- The class of regex formulas α with a *key attribute*, that is, a variable $x \in \text{Vars}(\alpha)$ with the property that for all strings s and tuples μ and μ' in $\llbracket \alpha \rrbracket(s)$, if $\mu(x) = \mu'(x)$ then $\mu = \mu'$.

A key attribute implies a polynomial bound as the number of spans of a string s is quadratic in $|s|$. Interestingly, its existence can be tested in polynomial time.

PROPOSITION 3.6. *Given a functional vset-automaton A with n states, and a variable $x \in \text{Vars}(A)$, it can be decided in time $O(n^4)$ whether x is a key attribute.*

3.3.3 Compilation to Automata. We now discuss the second evaluation approach: compiling the regex UCQ to a functional vset-automaton, and then applying Theorem 3.3. An immediate consequence of a combination with past results is as follows. There are computable (but potentially exponential) conversions of spanners in a regular representation into a vset-automaton (Fagin et al. [12]), and of a vset-automaton into a functional vset-automaton (Freydenberger [15]). Hence, we get the following.

COROLLARY 3.7. *Spanners q in a regular representation (e.g., regex UCQs) can be evaluated with FPT delay for the parameter $|q|$.*

The corollary should be contrasted with the traditional relational case, where Boolean CQ evaluation is W[1]-hard when the size of the query is the parameter [30]. Hence, in that respect regex UCQ evaluation is substantially more tractable than UCQ evaluation in the relational model.

³We could also use other yardsticks of enumeration efficiency, such as polynomial delay and *incremental polynomial time* [23]. Then, every occurrence of “polynomial total time” in this part would be replaced with the other yardstick.

Our next results are established by applying *efficient* compilations. Such compilations were obtained independently by Morciano et al. [28]; we discuss the differences in the approaches after each result (generally, both here and in [28], the proofs are based on standard constructions, but ref-words allow us to take shortcuts). Furthermore, our proofs also discuss the constructions with respect to Theorem 3.3. We begin with the most straightforward result, the projection operator.

LEMMA 3.8. *Given a functional vset-automaton A and $Y \subseteq \text{Vars}(A)$, one can construct in linear time a functional vset-automaton A_Y with $\llbracket A_Y \rrbracket = \llbracket \pi_Y(A) \rrbracket$.*

This is shown by replacing all transitions for operations on variables that are not in Y with ϵ -transitions. One advantage of our proof is that it showcases a nice use of the ref-word semantics. The situation is similar for the union operator.

LEMMA 3.9. *Given functional vset-automata $A_1, \dots, A_k$ such that $\text{Vars}(A_1) = \dots = \text{Vars}(A_k)$, one can construct in linear time a functional vset-automaton A with $\llbracket A \rrbracket = \llbracket A_1 \cup \dots \cup A_k \rrbracket$.*

Like [28], we prove this using the union construction for NFAs. In the proof, we also argue that the upper-bound for the worst case complexity of Theorem 3.3 is lower than the number of states of the constructed automaton suggests. Observe that in Lemma 3.9, the number of automata is not bounded. The situation is different for the join operator, which also uses the only construction that is not completely straightforward.

LEMMA 3.10. *Given two functional vset-automata A_1 and A_2 , each with $O(n)$ states and $O(v)$ variables, one can construct in time $O(vn^4)$ a functional vset-automaton A with $\llbracket A_1 \bowtie A_2 \rrbracket = \llbracket A \rrbracket$.*

Both this proof and the one from [28] build on the standard construction for automata intersection; the key difference is that our proof takes advantage of variable configurations. Note that joining k automata leads to a time of $O(vn^{2k})$, which is only polynomial if k is bounded. Due to Theorem 3.2, this is unavoidable under standard complexity theoretic assumptions.

This motivates the following definition. Let $k \geq 0$ be fixed. A *regex k -CQ* is a regex CQ with at most k atoms. A *regex k -UCQ* is a UCQ where each CQ is a k -CQ (i.e., a disjunction of k -CQs). From Lemmas 3.4 and 3.10 we conclude that we can convert, in polynomial time, a join of k regex formulas into a single functional vset-automaton. Then, Lemma 3.8 implies that projection can also be efficiently pushed into the functional vset-automaton. Hence, in polynomial time we can translate a k -CQ into a functional vset-automaton. Then, with Lemma 3.9 we conclude that this translation extends to k -UCQs. Finally, by applying Theorem 3.3 we arrive at the following main result.

THEOREM 3.11. *For every fixed k , regex k -UCQs can be evaluated with polynomial delay.*

Hence, where Theorem 3.11 applies, it is more powerful than Theorem 3.5: The former holds for all regex k -UCQs, while the latter has additional requirements, even when limited to k -UCQs.

In the next section we describe the algorithm of Theorem 3.3. Then, in Section 5 we extend the main theorems of this section, Theorems 3.5 and 3.11, to incorporate string equalities.

4 EVALUATING VSET-AUTOMATA

In this section, we give a high-level description of the proof of Theorem 3.3 and the resulting algorithm that, given a functional vset-automaton A and a string $s \in \Sigma^*$, enumerates $\llbracket A \rrbracket(s)$ with polynomial delay. Before we discuss the algorithm in Section 4.2, we first introduce the central notion of variable configurations.

4.1 Sequences of Variable Configurations

Before we present the actual algorithm, we first examine a property of functional vset-automata. Building on this, we introduce a novel way of working with vset-automata, which is the main part of our algorithm.

In order to introduce the concept, we consider an arbitrary functional vset-automaton $A = (V, Q, q_0, q_f, \delta)$, and ensure that all states are reachable from q_0 , and that q_f is reachable from every state.

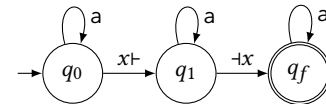
Then, for each state $q \in Q$ and all $x \in V$, each ref-word $\mathbf{r} \in (\Gamma_V \cup \Sigma)^*$ that takes A from q_0 to q satisfies exactly one of these mutually exclusive conditions:

- (1) $|\mathbf{r}|_{x\vdash} = |\mathbf{r}|_{\neg x} = 0$,
- (2) $|\mathbf{r}|_{x\vdash} = 1$ and $|\mathbf{r}|_{\neg x} = 0$, or
- (3) $|\mathbf{r}|_{x\vdash} = |\mathbf{r}|_{\neg x} = 1$, and $x\vdash$ occurs before $\neg x$.

If neither of these conditions is met, $\mathbf{r}$ contains one of $x\vdash$ or $\neg x$ more than once, or the two symbols appear in the wrong order. Then we can choose any ref-word $\mathbf{r}' \in (\Gamma_V \cup \Sigma)^*$ that takes A from q to q_f , and obtain a contradiction by observing that $\mathbf{r} \cdot \mathbf{r}' \in \mathcal{R}(A)$, although $\mathbf{r} \cdot \mathbf{r}'$ is not valid. Furthermore, if we compare two ref-words $\mathbf{r}_1$ and $\mathbf{r}_2$ that both take A from q_0 to a common state q , we know that both must satisfy the same of these three conditions (otherwise, $\mathbf{r}_1 \cdot \mathbf{r}'$ or $\mathbf{r}_2 \cdot \mathbf{r}'$ is not valid).

In other words, in each run of A , the information which variables have been opened or closed is stored implicitly in the states. To formalize this notion, we define the set $\mathcal{V}$ of *variable states* as $\mathcal{V} := \{w, o, c\}$ (the symbols stand for waiting, open, and closed). A *variable configuration* (for V) is a function $\vec{c}: V \rightarrow \mathcal{V}$. For each state $q \in Q$, we define its variable configuration $\vec{c}_q: V \rightarrow \mathcal{V}$ as follows: Choose any ref-word $\mathbf{r}$ that takes A from q_0 to q . We define $\vec{c}_q(x) := c$ if $\mathbf{r}$ contains $\neg x$, $\vec{c}_q(x) := o$ if $\mathbf{r}$ contains $x\vdash$ but not $\neg x$, and $\vec{c}_q(x) := w$ if $\mathbf{r}$ contains neither of the two symbols (as explained above, $\vec{c}_q$ is well-defined for each $q \in Q$).

EXAMPLE 4.1. Recall the functional vset-automaton A_{fun} from Example 2.6.



Then $\vec{c}_{q_0}(x) = w$, $\vec{c}_{q_1}(x) = o$, and $\vec{c}_{q_f}(x) = c$. ◇

Before we use this for the enumeration algorithm, we consider a more general view on variable configurations, that is independent of the automaton. As we shall see, the enumeration algorithm relies on the fact that for each $s \in \Sigma^*$ ($s = \sigma_1 \dots \sigma_N$ with $N \geq 0$), each (V, s) -tuple μ can be interpreted as a sequence of $N + 1$ variable configurations $\vec{c}_1, \dots, \vec{c}_{N+1}$ in the following way: For $x \in V$, assume that $\mu(x) = [i, j]$. For $1 \leq l \leq N + 1$, we define $\vec{c}_l(x) := w$

if $l < i$, $\vec{c}_l(x) := o$ if $i \leq l < j$, and $\vec{c}_l(x) := c$ if $l \geq j$. The idea is that each $\vec{c}_l$ is the variable configuration immediately before reading σ_l .

To illustrate this, consider a ref-word r with $\mu = \mu^r$ (see Section 2.2.1). Then $x \vdash$ is read between $\vec{c}_{i-1}$ and $\vec{c}_i$, while $\neg x$ is read between $\vec{c}_{j-1}$ and $\vec{c}_j$ (again ignoring the technicality that we do not define $\vec{c}_0$).

EXAMPLE 4.2. Let $V := \{x\}$, and let $s := aaa$. The following table contains all possible (V, s) -tuples, and the corresponding sequence $\vec{c}_1(x), \vec{c}_2(x), \vec{c}_3(x), \vec{c}_4(x)$:

$\mu(x)$	ref-word	$\vec{c}_1(x), \vec{c}_2(x), \vec{c}_3(x), \vec{c}_4(x)$
[1, 1]	$x \vdash \neg xaaa$	(c, c, c, c)
[1, 2]	$x \vdash a \neg x aa$	(o, c, c, c)
[1, 3]	$x \vdash aa \neg x a$	(o, o, c, c)
[1, 4]	$x \vdash aaa \neg x$	(o, o, o, c)
[2, 2]	$a x \vdash \neg xaa$	(w, c, c, c)
[2, 3]	$a x \vdash a \neg x a$	(w, o, c, c)
[2, 4]	$a x \vdash aa \neg x$	(w, o, o, c)
[3, 3]	$aa x \vdash \neg xa$	(w, w, c, c)
[3, 4]	$aa x \vdash a \neg x$	(w, w, o, c)
[4, 4]	$aaa x \vdash \neg x$	(w, w, w, c)

Note that this is exactly $\llbracket A_{\text{fun}} \rrbracket(s)$, where A_{fun} is the vset-automaton from Example 4.1. $\diamond$

We say that a sequence of variable configurations for V is *valid* if it respects the order of variables states; that is, $\vec{c}_i(x) = c$ implies $\vec{c}_{i+1}(x) = c$, and $\vec{c}_i(x) = o$ implies $\vec{c}_{i+1}(x) \in \{o, c\}$ for all $x \in V$. Obviously, each valid sequence of $|s| + 1$ variable configurations for V can be interpreted as a (V, s) -tuple; and it is easy to see that this is a one-to-one correspondence.

To connect this point of view to the variable configurations of A , note that each $r \in \text{Ref}(A, s)$ can be written as

$$r = r_0 \cdot \sigma_1 \cdot r_1 \cdot \sigma_2 \cdots r_{N-1} \cdot \sigma_N \cdot r_N,$$

where $r_i \in \Gamma_V^*$. For $1 \leq i \leq N + 1$, we determine $\vec{c}_i$ from the set $\{r_0 \cdot r_1 \cdots r_{i-1}\}$, as in the definition of the $\vec{c}_{q_i}$ above. This has the same effect as defining $\vec{c}_i := \vec{c}_{q_i}$ for any q_i that can be reached by processing $r_0 \cdot \sigma_1 \cdot r_1 \cdot \sigma_2 \cdots r_{i-1}$. In other words, in each run of A on r , immediately before processing σ_i , A must be in a state q_i with $\vec{c}_i = \vec{c}_{q_i}$. Thus, the sequence $\vec{c}_1, \dots, \vec{c}_{N+1}$ corresponds to the (V, s) -tuple μ^r .

Like ref-words, sequences of variable configurations can be understood as an abstraction of spanner behavior. In fact, both can be seen as successive steps of generalization: Ref-words hide the actual behavior of primitive spanner representations (i.e., the actual sequence of states in the vset-automaton, or which parts of the regex are mapped to which part of the input); they only express in which order variables are opened and closed. Sequences of variable configurations take this one step further, and compress successive variable operations (without terminals in-between) into a single step. Hence, treating $\llbracket A \rrbracket(s)$ as a language over the alphabet $\mathcal{V}^{|V|}$ is exactly the level of granularity that is needed to distinguish different tuples.

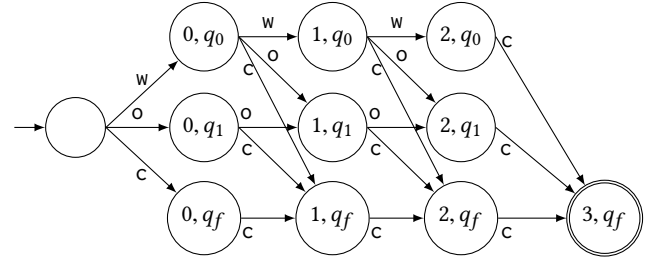


Figure 1: The NFA A_G from Example 4.3. For convenience, we write each variable configuration $\vec{c}_q$ as $\vec{c}_q(x)$.

Note that although some proofs in [15] use the concept of variable configurations for states, identifying (V, s) -tuples with sequences of variable configurations is a main conceptual contribution of the present paper.

4.2 The Algorithm

The algorithm enumerates the (V, s) -tuples of $\llbracket A \rrbracket(s)$ by enumerating the corresponding sequences of $|s| + 1$ variable configurations for V . In order to do so, we interpret each variable configuration as a letter of the alphabet $\mathcal{K} := \{\vec{c}_q \mid q \in Q\}$ (note that while there are $3^{|V|}$ possible letters that might occur in $\mathcal{K}$, its actual size is always bounded by $|Q|$). More specifically, the algorithm has the following two steps:

- (1) Given A and s , construct an NFA A_G over the alphabet $\mathcal{K}$ such that $\mathcal{L}(A_G)$ contains exactly those strings $\kappa_1 \cdots \kappa_{|s|+1}$, $\kappa_i \in \mathcal{K}$, that correspond to the elements of $\llbracket A \rrbracket(s)$.
- (2) Enumerate $\mathcal{L}(A_G)$ with polynomial delay.

The algorithm constructs the NFA A_G by first constructing a graph G whose nodes are tuples (i, q) , which encode that A can be in state q after reading $\sigma_1 \cdots \sigma_i$. The edges are drawn accordingly: There is an edge from (i, p) to $(i + 1, q)$ if A can reach q from p by reading σ_{i+1} and then arbitrarily many variable operations. The NFA A_G is then directly obtained from G by interpreting every edge from (i, p) to $(i + 1, q)$ as a transition for the letter $\vec{c}_q$.

Finally, to enumerate $\mathcal{L}(A_G)$ without repetitions, we tailor an optimized version of the algorithm by Ackerman and Shallit [2] that, given $l \geq 0$ and an NFA M over some alphabet T , enumerates $\mathcal{L}(M) \cap T^l$.

In summary, the graph G represents all runs of A on s . By interpreting (V, s) -tuples as strings from $\mathcal{K}^{|s|+1}$, we can treat G as an NFA A_G , and enumerate $\llbracket A \rrbracket(s)$ by enumerating $\mathcal{L}(A_G)$. As there is a one-to-one correspondence between these two sets, the fact that $\mathcal{L}(A_G)$ is enumerated without repetitions guarantees that no element of $\llbracket A \rrbracket(s)$ is repeated.

EXAMPLE 4.3. Consider A_{fun} (from Example 4.1) and $s := aaa$. From A_{fun} and s , the algorithm constructs the NFA A_G that is shown in Figure 1. Although we omit the step of constructing the graph G , note this can easily be obtained by removing the new starting state and all transition labels from A_G .

To see that $\mathcal{L}(A_G)$ corresponds to $\llbracket A_{\text{fun}} \rrbracket(s)$, take note that the table in Example 4.2 lists each element of $\llbracket A_{\text{fun}} \rrbracket(s)$ together with its corresponding sequence of variable configurations. $\diamond$

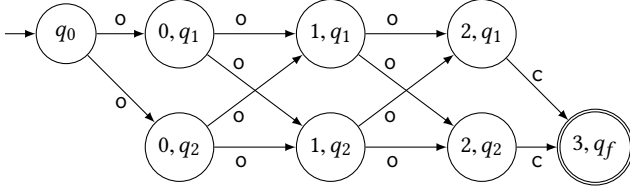
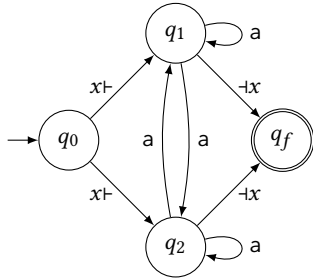


Figure 2: The NFA A_G from Example 4.4. We use the same notation as in Figure 1.

Note that the NFA A_G in Example 4.3 is deterministic, which means that enumerating $\mathcal{L}(A_G)$ actually requires less effort than in the general case: As stated in Theorem 3.3, if A has n states, we can enumerate $\llbracket A \rrbracket(s)$ with a delay of $O(n^2|s|)$. But if A_G is deterministic, this can be lowered to $O(|s|)$.

While there are automata where the worst case for the algorithm is reached, the stated upper bounds only applies to automata where the states have many outgoing transitions (or a large number of ϵ -transitions). For examples where we can use it for better upper bounds, see the proofs of Lemma 3.9 and Theorem 5.4. Next, we consider an example where A_G is non-deterministic.

EXAMPLE 4.4. In order to examine a case where constructing A_D is necessary, consider the following functional vset-automaton A :



It is easily seen that A is functional, and the variable configurations are defined by $\vec{c}_{q_0}(x) = w$, $\vec{c}_{q_1}(x) = \vec{c}_{q_2}(x) = o$, and $\vec{c}_{q_f}(x) = c$. Furthermore, for every $s \in a^*$, $\llbracket A \rrbracket(s)$ contains only a single μ , with $\mu(x) = [1, |s| + 1]$, which corresponds to the ref-word x^+s^-x . Nonetheless, for each such s , there are $2^{|s|}$ paths from q_0 to q_f in A . Now consider the case of $s = a^3$. By first constructing the graph G and then adding a new starting state q_0 , we obtain the NFA A_G that is shown in Figure 2.

Now, note that A_G is not deterministic, as $\vec{c}_{q_1} = \vec{c}_{q_2}$. In fact, $\mathcal{L}(A_G)$ consists only of the word $\vec{c}_{q_1} \cdot \vec{c}_{q_1} \cdot \vec{c}_{q_1} \cdot \vec{c}_{q_f}$, which corresponds to μ with $\mu(x) = [1, 4]$, the only element of $\llbracket A \rrbracket(s)$. $\diamond$

As a final remark, we observe that the algorithm uses the variable operations on the transitions of A only to compute the variable configurations. Afterwards, these transitions are treated as ϵ -transitions instead. This paper uses functional vset-automata to represent regex formulas. Instead, one could directly convert each regex formula into an ϵ -NFA A and a function that maps each state of A to a variable configuration. The enumeration algorithm and the “compilation lemmas” from Section 3.3.3 would directly work with this model. Whether this actually allows constructions that are more efficient or significantly simpler remains to be seen.

5 STRING EQUALITY

Our results in the previous two sections only apply to regex UCQs. As these are not allowed to use string equality selections, these have the same expressive power as regular spanner representations. In this section, we discuss how our results can be extended to include string equality selections, which allows us reach the full expressive power of core spanners.

5.1 Lower Bound

The main difficulty when dealing with string equality selections is that this operator quickly becomes computationally expensive, even without using joins. More specifically, Freydenberger and Holldack [16] showed that combined with string equalities, a single regex formula and a projection to a Boolean spanner already lead to an intractable evaluation problem.

THEOREM 5.1. [16] *Evaluation of Boolean regex CQs with string equalities is NP-complete, even if restricted to queries of the form $\pi_0 \zeta_{x_1, y_1}^= \cdots \zeta_{x_m, y_m}^= \alpha$.*

In other words, even a single regex formula already leads to NP-hardness. The proof from [16] uses a reduction from the membership problem for so-called pattern languages. It was shown by Fernau et al. [14] that this membership problem is W[1]-complete for various parameters. We prove that the situation is analogous for Boolean regex CQs with string equalities.

THEOREM 5.2. *Evaluation of Boolean regex CQs q with string equalities is W[1]-hard for the parameter $|q|$, even if restricted to queries of the form $\pi_0 \zeta_{x_1, y_1}^= \cdots \zeta_{x_m, y_m}^= \alpha$.*

PROOF. We prove the claim by modifying the proof of Theorem 3.2: Let s and γ be defined as in that proof. However, we do not use regex formulas δ_l to ensure that all variables $y_{i,l}$ and $x_{l,j}$ with $1 \leq i < l < j \leq k$ map to the same substring v_l . Instead, for each $1 \leq l \leq k$, we define a sequence S_l of $k-2$ binary string equality selections that is equivalent to the $(k-1)$ -ary string equality selection on the variables $y_{1,l}$ to $y_{l-1,l}$ and $x_{l,l+1}$ to $x_{l,k}$. We then define our query q as

$$q := \pi_0 S_1 \cdots S_k \gamma.$$

Note that being able to use string equality predicates, we do not need to iterate through all of the possible v as in the δ_l . Therefore the proof of correctness of the reduction used in the proof of Theorem 3.2 can be simply adapted to show correctness of this reduction. Observe that this is an FPT reduction with parameter $|q|$ since $|q|$ is of size $O(k^2)$ (i.e., the number of string equality predicates we use depends only on the clique size). Finally, we remark that like the proof of Theorem 3.2, this proof can be adapted to a binary alphabet by using the standard coding techniques. $\square$

Hence, while a regex CQ that consists of a single regex formula can be evaluated efficiently (due to Lemmas 3.4 and 3.8 and Theorem 3.3), even limited use of string equalities can become expensive. This result can be obtained by combining the reduction from [14] with the reduction from [16], the former proof requires encodings that are needlessly complicated for our purposes (this is caused by the comparatively low expressive power of pattern languages). Instead, we take the basic idea of an FPT-reduction from the k -clique

problem, and give a direct proof that is less technical. Note that the proof of Theorem 5.2 is different than that of Theorem 3.2 since in the former the query q constructed in the reduction is determined solely by the parameter k , and not the size of the graph as in the latter.

The reader should contrast Theorem 5.2 with Corollary 3.7. The combination of the two complexity results shows that, with respect to the parameter $|q|$, string equality considerably increases the parameterized complexity: Boolean regex CQ evaluation used to be FPT, and is now W[1]-hard.

Finally, note that queries as in Theorems 5.1 and 5.2 are always acyclic, as the variables of each equality atom also occur in α . Although they are not gamma acyclic, we can rewrite each query into an equivalent gamma-acyclic query with k -ary string equalities, by merging each pair ζ_X^- and ζ_Y^- with sets $X \cap Y \neq \emptyset$ into $\zeta_{X \cup Y}^-$ until all equalities range over pairwise disjoint sets.

5.2 Upper Bounds

We now examine the upper bounds for our two evaluation strategies on regex UCQs with string equalities. For the canonical relational approach, we observe that each equality atom corresponds to a relation that is of polynomial size. Hence, we can directly use Theorem 3.5 to conclude the following.

COROLLARY 5.3. *Let $\mathbf{P}$ be a class of regex UCQs with string equalities. If the regex formulas in the UCQs of $\mathbf{P}$ belong to a polynomially bounded class, and $\mathbf{P}$ maps to a tractable class of relational UCQs, then UCQs in $\mathbf{P}$ can be evaluated in polynomial total time.*

The upper bound for compilation to automata requires more effort. We first observe the following.

THEOREM 5.4. *For every fixed $m \geq 1$ there is an algorithm, that given a functional vset-automaton A , a sequence $S := \zeta_{x_1, y_1}^- \cdots \zeta_{x_m, y_m}^-$ of string equality selections over $\text{Vars}(A)$, and a string s , enumerates $\llbracket SA \rrbracket(s)$ with polynomial delay.*

The main idea of the proof is to construct a vset-automaton A_{eq} that defines exactly the string equalities on s . Specifically, $\mu \in \llbracket A_{\text{eq}} \rrbracket(s)$ holds if and only if $s_{\mu(x_i)} = s_{\mu(y_i)}$ for all x_i, y_i where ζ_{x_i, y_i}^- is a selection in S . We then use Lemma 3.10 to construct a vset-automaton A_{join} with $\llbracket A_{\text{join}} \rrbracket = \llbracket A \bowtie A_{\text{eq}} \rrbracket$, and use Theorem 3.3 to enumerate $\llbracket A_{\text{join}} \rrbracket(s) = \llbracket SA \rrbracket(s)$. Note that the construction of A_{eq} depends on s ; in fact, $\llbracket A_{\text{eq}} \rrbracket(s') = \emptyset$ for all strings $s' \neq s$. This dependency on s is unavoidable: Recall that as shown by Fagin et al. [12], regular spanners are strictly less expressive than core spanners (i. e., string equality adds expressive power). If we could construct an A_{eq} that worked on every string s' , this would immediately lead to a contradiction.

Analogously to the join in Section 3.3.3, the polynomial upper bound only holds for the construction if m is fixed, and Theorem 5.2 suggests that this cannot be overcome under standard assumptions. Hence, for all fixed $k, m \geq 0$, we define the notion of a *regex k -UCQ with up to m string equalities* analogously to a regex k -UCQ, with the additional requirement that each of the CQs uses at most m binary string equality selections. It follows from Theorem 5.4 that for every constant k , adding a fixed number of string equalities to

a regex k -UCQ does not affect its enumeration complexity (in the sense that the complexity stays polynomial).

COROLLARY 5.5. *For all fixed m and k , regex k -UCQs with up to m string equalities can be evaluated with polynomial delay.*

In other words, Theorem 3.11 can be extended to cover string equalities.

6 CONCLUDING REMARKS

We have studied the combined complexity of evaluating regex CQs and regex UCQs. We showed that the complexity is not determined only by the structure of the CQ as a relational query; rather, complexity can go higher since an atomic regex formula can already define a relation that is exponentially larger than the combined size of the input and output. Our upper bounds are based on an algorithm for evaluating a vset-automaton with polynomial delay. These bounds are based on two alternative evaluation strategies—the canonical relational evaluation and query-to-automaton compilation. We conclude the paper by proposing several directions for future research.

One direction for future research is to generalize the ability to compile into a vset-automaton to a class of queries that is richer than regex k -UCQs with string equality. In particular, we would like to have a robust definition of a class of algebraic expressions that we can efficiently translate into a vset-automaton.

Fagin et al. [12] showed that regular spanners, or equivalently regex UCQs (as we established in Theorem 2.8), can be phrased as *Unions of Conjunctive Regular Path Queries* (UCRPQs) [4, 6, 8, 9]. However, their translation entails an exponential blowup. Moreover, even if the translation could be made efficiently, it is not at all clear that tractability properties of the regex UCQ (e.g., bounded number of atoms, or low hypertree width) would translate into tractable properties of UCRPQs. Importantly, in the UCRPQ every atom involves two variables, and so, the problem of intractable materialization of an atom (i.e., the main challenge we faced here) does not occur. So, another future direction is a deeper exploration of the relationship between the complexity results of the two frameworks.

Last but not least, there is the crucial future direction of translating the upper bounds we presented into algorithms that substantially outperform the state of the art, at least when our tractability conditions hold. Beyond optimizing our translation and polynomial-delay algorithm, we would like to incorporate techniques of aggressive filtering for matching regular expressions [39, 40] and parallelizing polynomial-delay enumeration [40].

ACKNOWLEDGMENTS

The work of Dominik Freydenberger was supported by Deutsche Forschungsgemeinschaft (DFG) under grant FR 3551/1-1. The work of Benny Kimelfeld and Liat Peterfreund was supported by the Israel Science Foundation (ISF) Grant 1295/15, and by the German-Israeli Foundation for Scientific Research and Development (GIF), Grant I-2436-407.6/2016. The work of Liat Peterfreund was also supported by the Technion Hiroshi Fujiwara Cyber Security Research Center and the Israel Cyber Bureau, and by an IBM PhD Fellowship Award.

REFERENCES

- [1] Serge Abiteboul, Richard Hull, and Victor Vianu. 1995. *Foundations of Databases*. Addison-Wesley.
- [2] Margareta Ackerman and Jeffrey Shallit. 2009. Efficient enumeration of words in regular languages. *Theor. Comput. Sci.* 410, 37 (2009), 3461–3470.
- [3] Jitendra Ajmera, Hyung-Il Ahn, Meena Nagarajan, Ashish Verma, Danish Contractor, Stephen Dill, and Matthew Denesuk. 2013. A CRM system for social media: challenges and experiences. In *WWW*. ACM, 49–58.
- [4] Pablo Barceló, Miguel Romero, and Moshe Y. Vardi. 2016. Semantic Acyclicity on Graph Databases. *SIAM J. Comput.* 45, 4 (2016), 1339–1376.
- [5] Edward Benson, Aria Haghighi, and Regina Barzilay. 2011. Event Discovery in Social Media Feeds. In *ACL*. The Association for Computer Linguistics, 389–398.
- [6] Diego Calvanese, Giuseppe De Giacomo, Maurizio Lenzerini, and Moshe Y. Vardi. 2000. Containment of Conjunctive Regular Path Queries with Inverse. In *KR*. Morgan Kaufmann, 176–185.
- [7] Ashok K. Chandra and Philip M. Merlin. 1977. Optimal Implementation of Conjunctive Queries in Relational Data Bases. In *STOC*. ACM, 77–90.
- [8] Mariano P. Consens and Alberto O. Mendelzon. 1990. GraphLog: a Visual Formalism for Real Life Recursion. In *PODS*. ACM, 404–416.
- [9] Alin Deutsch and Val Tannen. 2001. Optimization Properties for Classes of Conjunctive Regular Path Queries. In *DBPL*. Springer, 21–39.
- [10] R. G. Downey and M. R. Fellows. 1999. *Parameterized Complexity*. Springer.
- [11] Ronald Fagin. 1983. Acyclic Database Schemes (of Various Degrees): A Painless Introduction. In *CAAP*. Springer, 65–89.
- [12] Ronald Fagin, Benny Kimelfeld, Frederick Reiss, and Stijn Vansummeren. 2015. Document Spanners: A Formal Approach to Information Extraction. *J. ACM* 62, 2 (2015), 12.
- [13] Ronald Fagin, Benny Kimelfeld, Frederick Reiss, and Stijn Vansummeren. 2016. Declarative Cleaning of Inconsistencies in Information Extraction. *ACM Trans. Database Syst.* 41, 1 (2016), 6.
- [14] Henning Fernau, Markus L. Schmid, and Yngve Villanger. 2016. On the Parameterised Complexity of String Morphism Problems. *Theory Comput. Syst.* 59, 1 (2016), 24–51.
- [15] Dominik D. Freydenberger. 2017. A Logic for Document Spanners. In *ICDT*. 13:1–13:18. Full version available at <http://ddfy.de/publications/F-ALFDS.html>.
- [16] Dominik D. Freydenberger and Mario Holldack. 2017. Document Spanners: From Expressive Power to Decision Problems. *Theory Comput. Syst.* (2017), 1–45.
- [17] Qiang Fu, Jian-Guang Lou, Yi Wang, and Jiang Li. 2009. Execution Anomaly Detection in Distributed Systems through Unstructured Log Analysis. In *ICDM*. IEEE Computer Society, 149–158.
- [18] Georg Gottlob, Zoltán Miklós, and Thomas Schwentick. 2009. Generalized hypertree decompositions: NP-hardness and tractable variants. *J. ACM* 56, 6 (2009).
- [19] Ralph Grishman and Beth Sundheim. 1996. Message Understanding Conference-6: A Brief History. In *COLING*. ACL, 466–471.
- [20] M. Grohe and J. Flum. 2006. *Parameterized Complexity Theory*. Springer.
- [21] Johannes Hoffart, Fabian M. Suchanek, Klaus Berberich, and Gerhard Weikum. 2013. YAGO2: A spatially and temporally enhanced knowledge base from Wikipedia. *Artif. Intell.* 194 (2013), 28–61.
- [22] John E. Hopcroft and Jeffrey D. Ullman. 1979. *Introduction to Automata Theory, Languages, and Computation*. Addison-Wesley.
- [23] David S. Johnson, Christos H. Papadimitriou, and Mihalis Yannakakis. 1988. On Generating All Maximal Independent Sets. *Inf. Process. Lett.* 27, 3 (1988), 119–123.
- [24] Rajasekar Krishnamurthy, Yunyao Li, Sriram Raghavan, Frederick Reiss, Shivakumar Vaithyanathan, and Huaiyu Zhu. 2008. SystemT: A system for declarative information extraction. *SIGMOD Record* 37, 4 (2008), 7–13.
- [25] Yunyao Li, Frederick Reiss, and Laura Chiticariu. 2011. SystemT: A Declarative Information Extraction System. In *ACL*. ACL, 109–114.
- [26] Bin Liu, Laura Chiticariu, Vivian Chu, H. V. Jagadish, and Frederick Reiss. 2010. Automatic Rule Refinement for Information Extraction. *PVLDB* 3, 1 (2010), 588–597.
- [27] Katja Losemann. 2015. *Foundations of Regular Languages for Processing RDF and XML*. Ph.D. Dissertation. University of Bayreuth. <https://epub.uni-bayreuth.de/2536/>
- [28] Andrea Morciano, Martin Ugarte, and Stijn Vansummeren. 2016. *Automata-Based Evaluation of AQL queries*. Technical Report. Université Libre de Bruxelles. http://db.ing.puc.cl/~mugarte/aql_eval.pdf
- [29] Yoav Nahshon, Liat Peterfreund, and Stijn Vansummeren. 2016. Incorporating information extraction in the relational database model. In *WebDB*. ACM, 6.
- [30] C. H. Papadimitriou and M. Yannakakis. 1999. On the Complexity of Database Queries. *J. Comput. Syst. Sci.* 58, 3 (1999), 407–427.
- [31] Frederick Reiss, Sriram Raghavan, Rajasekar Krishnamurthy, Huaiyu Zhu, and Shivakumar Vaithyanathan. 2008. An Algebraic Approach to Rule-Based Information Extraction. In *ICDE*. IEEE Computer Society, 933–942.
- [32] Christopher De Sa, Alexander Ratner, Christopher Ré, Jaeho Shin, Feiran Wang, Sen Wu, and Ce Zhang. 2016. DeepDive: Declarative Knowledge Base Construction. *SIGMOD Record* 45, 1 (2016), 60–67.
- [33] Markus L. Schmid. 2016. Characterising REGEX Languages by Regular Languages Equipped with Factor-Referencing. *Inform. Comput.* 249 (2016), 1–17.
- [34] Warren Shen, AnHai Doan, Jeffrey F. Naughton, and Raghu Ramakrishnan. 2007. Declarative Information Extraction Using Datalog with Embedded Extraction Predicates. In *VLDB*. ACM, 1033–1044.
- [35] Jaeho Shin, Sen Wu, Feiran Wang, Christopher De Sa, Ce Zhang, and Christopher Ré. 2015. Incremental Knowledge Base Construction Using DeepDive. *PVLDB* 8, 11 (2015), 1310–1321.
- [36] Fabian M. Suchanek, Gjergji Kasneci, and Gerhard Weikum. 2007. YAGO: A core of semantic knowledge. In *WWW*. ACM, 697–706.
- [37] Daniel J. Walker, David E. Clements, Maki Darwin, and Jan W. Amtrup. 2001. Sentence boundary detection: A comparison of paradigms for improving MT quality. In *MT Summit VIII*. EAMT, 18–22.
- [38] Hua Xu, Shane P. Stenner, Son Doan, Kevin B. Johnson, Lemuel R. Waitman, and Joshua C. Denny. 2010. MedEx: a medication information extraction system for clinical narratives. *JAMIA* 17, 1 (2010), 19–24.
- [39] Xiaochun Yang, Tao Qiu, Bin Wang, Baihua Zheng, Yaoshu Wang, and Chen Li. 2016. Negative Factor: Improving Regular-Expression Matching in Strings. *ACM Trans. Database Syst.* 40, 4 (2016), 25.
- [40] Xiaochun Yang, Bin Wang, Tao Qiu, Yaoshu Wang, and Chen Li. 2013. Improving regular-expression matching on strings using negative factors. In *SIGMOD*. ACM, 361–372.
- [41] Mihalis Yannakakis. 1981. Algorithms for Acyclic Database Schemes. In *VLDB*. IEEE Computer Society, 82–94.
- [42] Alexander Yates, Michele Banko, Matthew Broadhead, Michael J. Cafarella, Oren Etzioni, and Stephen Soderland. 2007. TextRunner: Open Information Extraction on the Web. In *ACL-HLT*. ACL, 25–26.
- [43] Huaiyu Zhu, Sriram Raghavan, Shivakumar Vaithyanathan, and Alexander Löser. 2007. Navigating the intranet with high precision. In *WWW*. ACM, 491–500.